Hes·so VALAIS WALLIS
Haute Ecole d'Ingénierie
Hochschule für Ingenieurwissenschaften

Nom : _____

Prénom : _____

Classe : _____

SOLUTIONS EN ROUGE

Examen intermédiaire 1 – Embedded systems 1 - Embedded programming

Question / points	1 / 3	2 / 3	3 / 10	4/6	5 / 14	Total / 36
Résultat						

Consigne :

Lisez attentivement la donnée et répondez de manière **lisible** aux questions. Le cours et la documentation de laboratoire sont autorisés.

Question 1 (le PIC): (3 points)

Répondez par **VRAI** ou **FAUX**.

Sur le PIC18, le TIMER2 peut être utilisé avec une clock externe ? FAUX

L'écriture d'un bit sur le port x doit se faire avec le registre LATx ? VRAI

Le PIC18 permet de faire un PWM avec une résolution de 10 bits ? VRAI

Pour que le TIMER1 fasse des interruptions, vous devez mettre à 1 : TMR1IE, et GIE? FAUX

Lorsque le flag GIE est à 0, seule l'interruption de RB0 (INT0) est traitée ? FAUX

Le PIC18 est un processeur von Neumann ? FAUX

Question 2 (le C): (3 points)

Répondez par **VRAI** ou **FAUX**.

La variable `uint8_t MyCounter` respecte les conventions CamelCase ? FAUX

Une variable locale est toujours initialisée à 0 ? FAUX

Une variable globale doit être placée dans un fichier « `.h` » ? FAUX

Le mot clé `static` pour une variable locale rend cette variable visible aux autres fonctions ? FAUX

Un tableau en 'C' est toujours passé par valeur dans une fonction ? FAUX

Une structure en 'C' est toujours passé par référence dans une fonction ? FAUX

Question 3: (10 points)

Ecrivez une fonction en C qui va inverser la position des éléments d'un tableau.
Ci-dessous, l'appel à la fonction. Attention, la taille du tableau n'est pas définie ainsi que la taille de l'élément.

```
uint16_t table[] = {33,54,12,8,2,93,124,91};

void main(void)
{
  ...
  SwapTable(table, sizeof(table), sizeof(table[0])) ;
  ...
}
```

```
void SwapTable(void * tbl, uint32_t tblSize, uint32_t elemSize)
{
// complex solution
   uint8_t tmp;
   uint8_t count = tblSize/elemSize;
   uint8_t * ptr8 = tbl;

   for(uint32_t i=0 ; i<count/2 ; i++)
   {
       for(uint8_t j=0;j<elemSize;j++)
       {
           tmp = ptr8[i*elemSize + j];
           ptr8[i*elemSize + j] = ptr8[tblSize-i*elemSize -
                                  (elemSize-1)+j-1];
           ptr8[tblSize-i*elemSize-(elemSize-1)+j-1] = tmp;
       }
   }

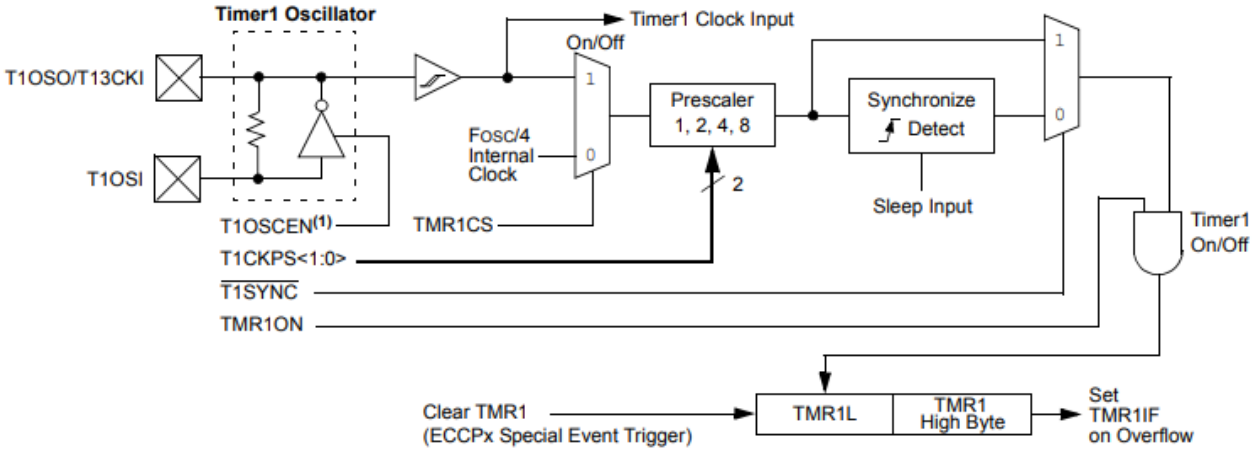
// easy solution
   uint32_t tmp;
   uint8_t elemCount = tblSize/elemSize;
   for(uint32_t i=0 ; i<elemCount/2 ; i++)
   {
       memcpy(&tmp,
              &ptr[i*elemSize],
              elemSize);
       memcpy(&ptr[i*elemSize],
              &ptr[elemCount*elemSize-i*elemSize]-elemSize,
              elemSize);
       memcpy&ptr[elemCount*elemSize-i*elemSize]-elemSize,
              tmp,
              elemSize);
   }

}  // utilisez une feuille supplémentaire si nécessaire
```

Question 4: (12 points)

On veut mesurer une fréquence entre 10 Hz et 20 Hz avec la meilleure résolution possible. Pour cela, nous connectons ce signal sur l'entrée d'interruption RB0. Le timer 1 nous servira à compter le temps entre deux impulsions.
La fréquence de l'oscillateur du microcontrôleur est de 4MHz.
Sur la figure ci-dessous, veuillez entourer les réglages choisis pour le timer 1 (TMR1CS, prescaler, T1SYNC).

T1SYNC (any), prescaler = 2, TMR1CS = 0



Complétez le code de l'interruption RB0 pour que la variable freqmhz contienne la fréquence mesurée en milihertz. Attention, l'utilisation de variables de type *float* ou *double* est interdit.

```
uint16_t freqmhz;
void _interrupt() MyISR(void)
{

  if(INT0IE == 1) && (INT0IF == 1)
  {
      INT0IF = 0;
      Freqmhz = 2000000 / TMR1;
      TMR1 = 0;

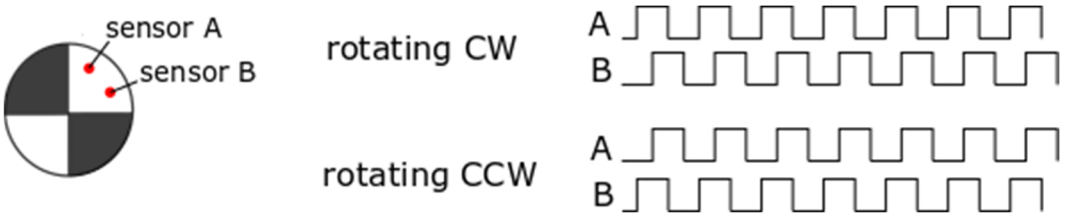


  }
}
```

On peut faire encore mieux en utilisant les interruption du Timer1 !!

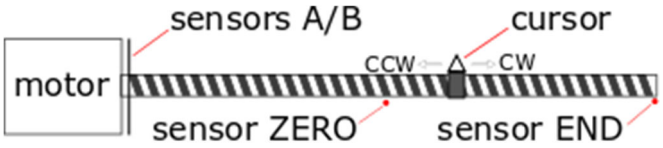
Question 5: (14 points)

Soit un capteur optique placé sur l'axe d'un moteur selon la figure suivante. Lorsque le capteur est sur le noir, la valeur 1 est retournée. Quand il est sur le blanc, c'est la valeur 0 qui sera lue.



Deux signaux sont à disposition du microcontrôleur. Le signal A et le signal B permettent de déterminer le nombre de tour du moteur ainsi que son sens de rotation.

On désire en tout temps connaitre **le nombre de tours effectués** dans un sens ou dans l'autre par rapport à la position 0 (capteur de position fixe appelé ZERO qui se trouve au centre du dispositif présenté plus loin). La variable `pos` contiendra la valeur absolue du nombre de tours du moteur (nombre de tours en positif ou négatif). La valeur `posCursor` contiendra la position du curseur en millimètre par rapport au point 0 sachant qu'un tour de moteur correspond à un déplacement de 0.5mm sur la vis sans fin.



Choisissez judicieusement les pins sur lesquelles seront connectés ces signaux sachant que l'on désire naturellement utiliser les interruptions au mieux.

SENSOR_A = pin RB0

SENSOR_B = pin RB1

Quelle pourrait être la résolution **maximale** théorique en um de la position du curseur (indépendamment de votre code) ? : 0.5mm / 8 => 62.5 um

Si la variable `pos` est une variable signée 8 bits, quel sera la longueur maximale de la vis sans fin pour ne pas avoir de dépassement ? : 127 * 0.5mm => 63.5 mm de chaque coté

Observez bien le dessin du dispositif. Ecrivez la fonction `InitPos()` qui doit faire l'initialisation du système lors de la mise sous tension à l'aide de la Commande `Drive(int8_t direction)` et de la lecture des capteurs? :

Le paramètre de la commande Drive peut prendre 3 valeurs :

- 0 = moteur arrêté
- 1 = moteur en direction CW
- -1 = moteur en direction CCW

La fonction `InitPos()` effectuera ce travail.

```
void InitPos(void)
{
    Drive(1);
    While(sensorEnd == 0){}
    Drive(-1);
    While(sensorZero == 0){}
    Drive(0);
    pos = 0;
    posCursor = 0;









}
```

Sur la feuille suivante, écrivez le code en 'C' réalisant ce programme :

- fonction `main()` initialisations des interruptions avant appel de `InitPos()` et du `while(1)`
- fonction d'interruptions gérant la position

```
int8_t pos;

int8_t posCursor;

void main(void)                 // this is the main of question 5
{

    INT0IE = 1;
    INTEDG0 = "rising";     // 0 or 1, need to read datasheet
    GIE = 1;

  InitPos();
  while(1)
  {
  }
}
```

```
void __interrupt() AnyISR(void)   // this is the interrupt of question 5
{
  static int8_t isrCount=0;
  INT0IF = 0;
  if(RB1 == 0)
  {
    isrCount++;
    if(isrCount == 2)
    {
      isrCount=0;
      pos++;
    }
  }
  else
  {
    isrCount--;
    if(isrCount == -2)
    {
      isrCount=0;
      pos--;
    }
  }
  posCursor = pos / 2;









}
```